

UNIVERSIDAD DA VINCI DE GUATEMALA
FACULTAD DE INGENIERÍA
CARRERA: LICENCIATURA EN INGENIERÍA EN SISTEMAS



**LOREM IPSUM IS SIMPLY DUMMY TEXT OF THE PRINTING AND
TYPESETTING.**

Proyecto Final

Miguel Estuardo Fuentes Monzón

Carne 202301585

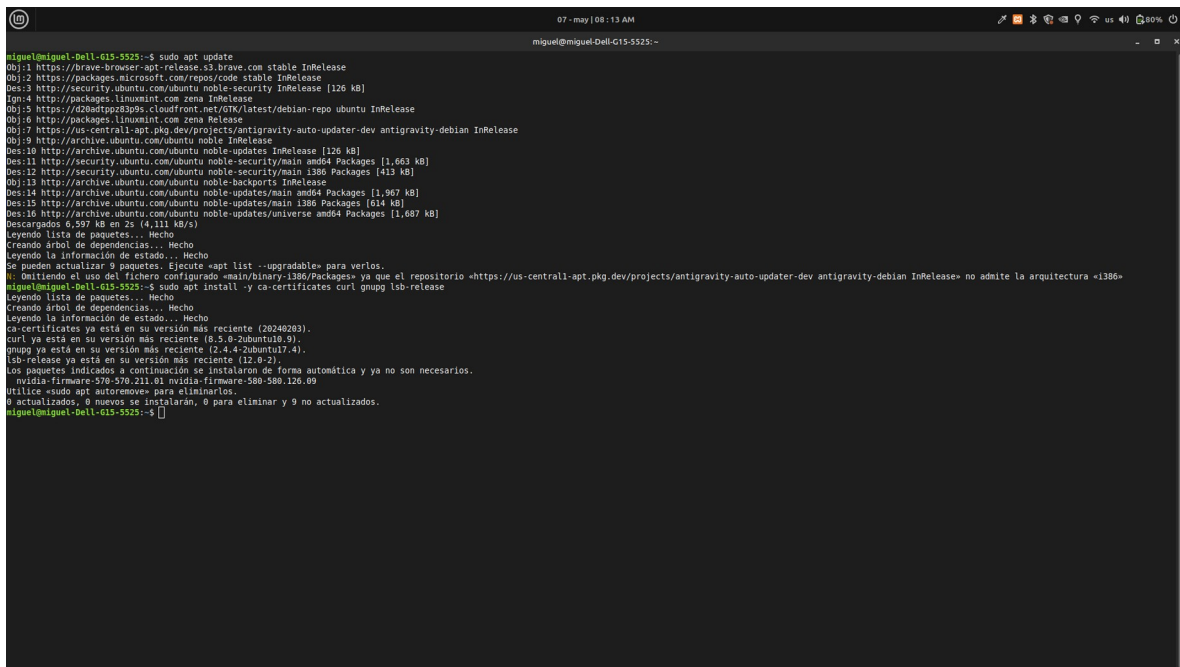
Guatemala, 07 de Mayo de 2026.

Procedimiento

Paso 1 — Instalar Docker

Instalar dependencias:

- `sudo apt update`
- `sudo apt install -y ca-certificates curl gnupg lsb-release`



```
miguel@miguel-Dell-G15-5525:~$ sudo apt update
07 - may 08:13 AM
miguel@miguel-Dell-G15-5525:~$ sudo apt update
Obj:1 https://brave-browser-apt-release.s3.brave.com stable InRelease
Obj:2 https://packages.microsoft.com/repos/code stable InRelease
Des:3 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Ign:4 http://packages.linuxmint.com zena InRelease
Obj:5 https://d2badtpz283pqs.cloudfront.net/OTK/latest/debian-repo ubuntu InRelease
Obj:6 http://packages.linuxmint.com zena release
Obj:7 https://us-central1-apt.pkg.dev/projects/antigravity-auto-updater-dev antigravity-debian InRelease
Obj:9 https://archive.ubuntu.com/ubuntu noble InRelease
Des:10 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Des:11 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1,663 kB]
Des:12 http://security.ubuntu.com/ubuntu noble-security/main i386 Packages [413 kB]
Obj:13 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Des:14 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1,967 kB]
Des:15 http://archive.ubuntu.com/ubuntu noble-updates/main i386 Packages [614 kB]
Des:16 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1,687 kB]
Descargados 6,597 kB en 2s (4,111 kB/s)
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se pueden actualizar 9 paquetes. Ejecute 'apt list --upgradable' para verlos.
W: Omitiendo el uso del fichero configurado 'main/binary-i386/Packages' ya que el repositorio 'https://us-central1-apt.pkg.dev/projects/antigravity-auto-updater-dev antigravity-debian InRelease' no admite la arquitectura 'i386'
miguel@miguel-Dell-G15-5525:~$ sudo apt install -y ca-certificates curl gnupg lsb-release
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
ca-certificates ya está en su versión más reciente (20240203).
curl ya está en su versión más reciente (8.5.0-2ubuntu10.9).
gnupg ya está en su versión más reciente (2.4.4-2ubuntu17.4).
lsb-release ya está en su versión más reciente (12.0-2).
Los paquetes indicados a continuación se instalaron de forma automática y ya no son necesarios.
nvidia-firmware-570-570.211.01 nvidia-firmware-580-580.126.09
Utilice 'sudo apt autoremove' para eliminarlos.
0 actualizados, 0 nuevos se instalarán, 0 para eliminar y 9 no actualizados.
miguel@miguel-Dell-G15-5525:~$
```

Agregar la llave GPG oficial de Docker:

- `sudo install -m 0755 -d /etc/apt/keyrings`
- `curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg`
- `sudo chmod a+r /etc/apt/keyrings/docker.gpg`

```
miguel@miguel-Dell-G15-5525:~$ sudo install -m 0755 -d /etc/apt/keyrings
miguel@miguel-Dell-G15-5525:~$ curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
miguel@miguel-Dell-G15-5525:~$ sudo chmod a+r /etc/apt/keyrings/docker.gpg
miguel@miguel-Dell-G15-5525:~$
```

Agregar el repositorio de Docker:

echo \

"deb [arch=\$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu \

jammy stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

```
miguel@miguel-Dell-G15-5525:~$ echo \
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu \
jammy stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
miguel@miguel-Dell-G15-5525:~$
```

Instalar Docker:

- sudo apt update
- sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin

```
07 - may | 08:17 AM
miguel@miguel-Dell-G15-5525:~$ sudo apt update
containerd.io docker-buildx-plugin docker-ce docker-ce-cli containerd.io docker-compose-plugin libslirp0 pigz slirp4netns
0 actualizados, 9 nuevos se instalarán, 0 para eliminar y 9 no actualizados.
Se necesita descargar 94.9 MB de archivos.
Se utilizarán 365 MB de espacio de disco adicional después de esta operación.
Des-1 https://archive.ubuntu.com/ubuntu noble/universe amd64 pigz amd64 2.8-1 [65.6 kB]
Des-2 https://download.docker.com/linux/ubuntu jammy/stable amd64 containerd.io amd64 2.2.3-1-ubuntu.22.04-jammy [23.6 MB]
Des-3 https://archive.ubuntu.com/ubuntu noble/main amd64 libslirp0 amd64 4.7.0-1ubuntu2 [63.8 kB]
Des-4 https://archive.ubuntu.com/ubuntu noble/universe amd64 slirp4netns amd64 1.2.1-1build1 [54.9 kB]
Des-5 https://download.docker.com/linux/ubuntu jammy/stable amd64 docker-ce-cli amd64 5:29.4.3-1-ubuntu.22.04-jammy [17.0 MB]
Des-6 https://download.docker.com/linux/ubuntu jammy/stable amd64 docker-ce amd64 5:29.4.3-1-ubuntu.22.04-jammy [22.0 MB]
Des-7 https://download.docker.com/linux/ubuntu jammy/stable amd64 docker-buildx-plugin amd64 0.33.0-1-ubuntu.22.04-jammy [16.9 MB]
Des-8 https://download.docker.com/linux/ubuntu jammy/stable amd64 docker-ce-rootless-extras amd64 5:29.4.3-1-ubuntu.22.04-jammy [6.559 kB]
Des-9 https://download.docker.com/linux/ubuntu jammy/stable amd64 docker-compose-plugin amd64 5.1.3-1-ubuntu.22.04-jammy [8,023 kB]
Descargados 94.9 MB en 2s (50.7 MB/s)
Seleccionando el paquete containerd.io previamente no seleccionado.
(Leyendo la base de datos ... 60713 ficheros o directorios instalados actualnente.)
Preparando para desempaquetar .../0-containerd.io (2.2.3-1-ubuntu.22.04-jammy amd64 deb ...)
Desempaquetando containerd.io (2.2.3-1-ubuntu.22.04-jammy) ...
Seleccionando el paquete docker-ce-cli previamente no seleccionado.
Preparando para desempaquetar .../1-docker-ce-cli 5%3a29.4.3-1-ubuntu.22.04-jammy amd64 deb ...
Desempaquetando docker-ce-cli (5:29.4.3-1-ubuntu.22.04-jammy) ...
Seleccionando el paquete docker-ce previamente no seleccionado.
Preparando para desempaquetar .../2-docker-ce 5%3a29.4.3-1-ubuntu.22.04-jammy amd64 deb ...
Desempaquetando docker-ce (5:29.4.3-1-ubuntu.22.04-jammy) ...
Seleccionando el paquete pigz previamente no seleccionado.
Preparando para desempaquetar .../3-pigz 2.8-1 amd64 deb ...
Desempaquetando pigz (2.8-1) ...
Seleccionando el paquete docker-buildx-plugin previamente no seleccionado.
Preparando para desempaquetar .../4-docker-buildx-plugin 0.33.0-1-ubuntu.22.04-jammy amd64 deb ...
Desempaquetando docker-buildx-plugin (0.33.0-1-ubuntu.22.04-jammy) ...
Seleccionando el paquete docker-ce-rootless-extras previamente no seleccionado.
Preparando para desempaquetar .../5-docker-ce-rootless-extras 5%3a29.4.3-1-ubuntu.22.04-jammy amd64 deb ...
Desempaquetando docker-ce-rootless-extras (5:29.4.3-1-ubuntu.22.04-jammy) ...
Seleccionando el paquete docker-compose-plugin previamente no seleccionado.
Preparando para desempaquetar .../6-docker-compose-plugin 5.1.3-1-ubuntu.22.04-jammy amd64 deb ...
Desempaquetando docker-compose-plugin (5.1.3-1-ubuntu.22.04-jammy) ...
Seleccionando el paquete libslirp0 previamente no seleccionado.
Preparando para desempaquetar .../7-libslirp0 4.7.0-1ubuntu3 amd64 deb ...
Desempaquetando libslirp0 amd64 (4.7.0-1ubuntu3) ...
Seleccionando el paquete slirp4netns previamente no seleccionado.
Preparando para desempaquetar .../8-slirp4netns 1.2.1-1build2 amd64 deb ...
Desempaquetando slirp4netns (1.2.1-1build2) ...
Configurando docker-buildx-plugin (0.33.0-1-ubuntu.22.04-jammy) ...
Configurando containerd.io (2.2.3-1-ubuntu.22.04-jammy) ...
Created symlink /etc/systemd/system/multi-user.target.wants/containerd.service → /usr/lib/systemd/system/containerd.service.
Configurando docker-compose-plugin (5.1.3-1-ubuntu.22.04-jammy) ...
Configurando docker-ce-cli (5:29.4.3-1-ubuntu.22.04-jammy) ...
Configurando libslirp0 amd64 (4.7.0-1ubuntu3) ...
Configurando pigz (2.8-1) ...
Configurando docker-ce-rootless-extras (5:29.4.3-1-ubuntu.22.04-jammy) ...
Configurando slirp4netns (1.2.1-1build2) ...
Configurando docker-ce (5:29.4.3-1-ubuntu.22.04-jammy) ...
Created symlink /etc/systemd/system/multi-user.target.wants/docker.service → /usr/lib/systemd/system/docker.service.
Created symlink /etc/systemd/system/sockets.target.wants/docker.socket → /usr/lib/systemd/system/docker.socket.
Procesando disparadores para man-db (2.12.0-4build2) ...
Procesando disparadores para libc-bin (2.39-0ubuntu8.7) ...
miguel@miguel-Dell-G15-5525:~$
```

Verificar que funcionó:

- sudo docker run hello-world

```
07 - may | 08:17 AM
miguel@miguel-Dell-G15-5525:~$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4550667f608: Pull complete
0678642a375: Download complete
Digest: sha256:f9078146db2e5e784360b1fe3841ee8317f44827d1bf7da0b5279626805
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
miguel@miguel-Dell-G15-5525:~$
```

Paso 2 — Instalar Jenkins (dentro de Docker)

Crear una carpeta para Jenkins:

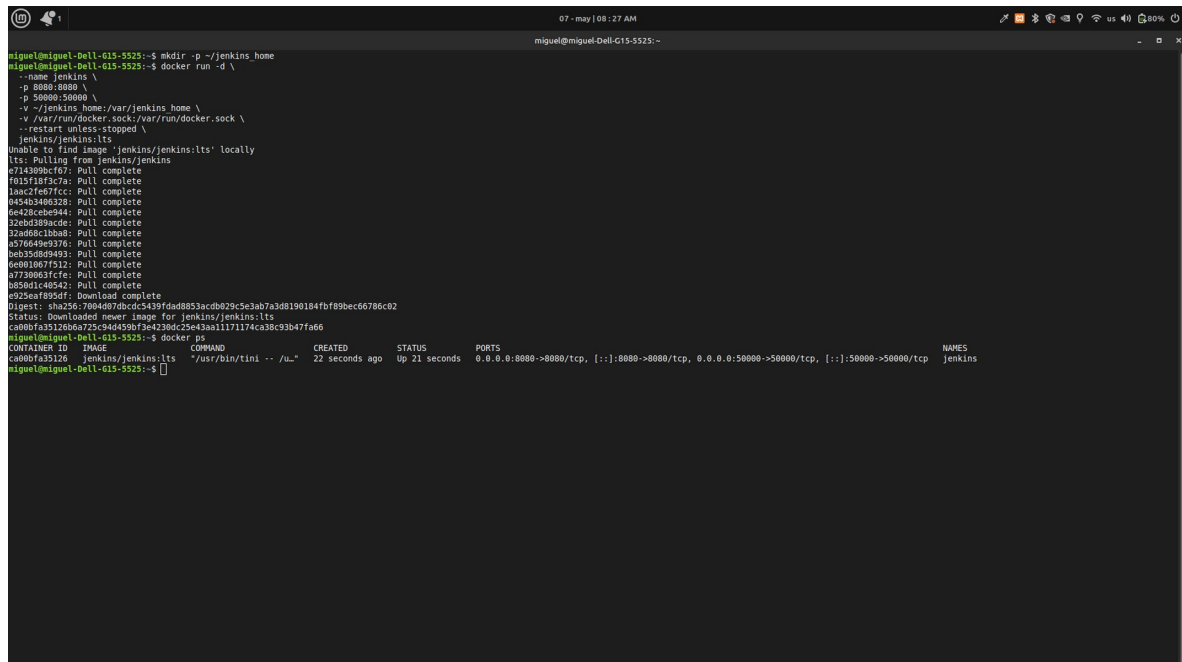
- `mkdir -p ~/jenkins_home`

Levanta Jenkins:

```
docker run -d \  
  
--name jenkins \  
  
-p 8080:8080 \  
  
-p 50000:50000 \  
  
-v ~/jenkins_home:/var/jenkins_home \  
  
-v /var/run/docker.sock:/var/run/docker.sock \  
  
--restart unless-stopped \  
  
jenkins/jenkins:lts
```

Verificar que esté corriendo:

- `docker ps`

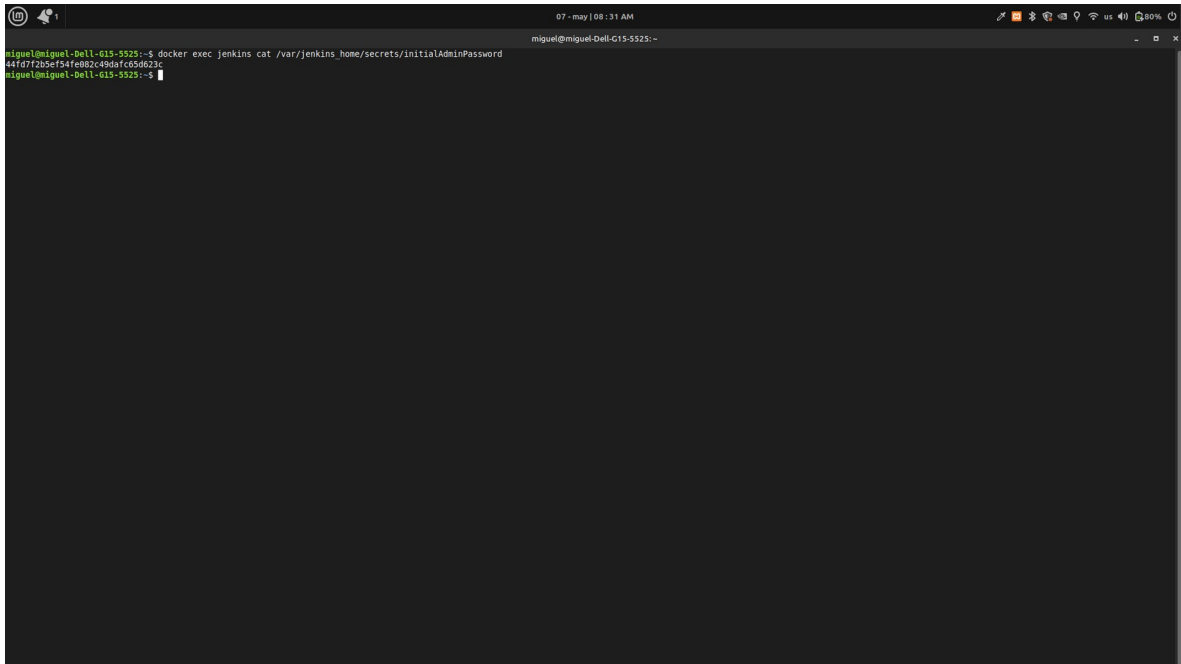


```
miguel@miguel-Dell-G15-5525:~$ mkdir -p ~/jenkins_home
miguel@miguel-Dell-G15-5525:~$ docker run -d \
  --name jenkins \
  -p 8080:8080 \
  -p 50000:50000 \
  -v ~/jenkins_home:/var/jenkins_home \
  -v /var/run/docker.sock:/var/run/docker.sock \
  --restart unless-stopped \
  jenkins/jenkins:lts
Unable to find image 'jenkins/jenkins:lts' locally
lts: Pulling from jenkins/jenkins
e714399bcf67: Pull complete
1035f1813c7a: Pull complete
1aac2fe07fcc: Pull complete
0454b3406328: Pull complete
6e032cebe944: Pull complete
32abdb389acde: Pull complete
32ad68c1bba8: Pull complete
a576649e2d76: Pull complete
beb350b09493: Pull complete
6e001007f532: Pull complete
a77300e3fcef: Pull complete
b050d1c40542: Pull complete
a925eaf895df: Download complete
Digest: sha256:7094d07b0dc5439fdad8053acdb029c5e3ab7a3d8190184fbf89bec66786c02
Status: Downloaded newer image for jenkins/jenkins:lts
ca80bfa35126b6a725c94d4590f3e4238dc25e43aa1171174ca38c93b47fa66
miguel@miguel-Dell-G15-5525:~$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
ca80bfa35126	jenkins/jenkins:lts	"/usr/bin/tini -- /u..."	22 seconds ago	Up 21 seconds	0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 0.0.0.0:50000->50000/tcp, [::]:50000->50000/tcp	jenkins

Obtener la contraseña inicial:

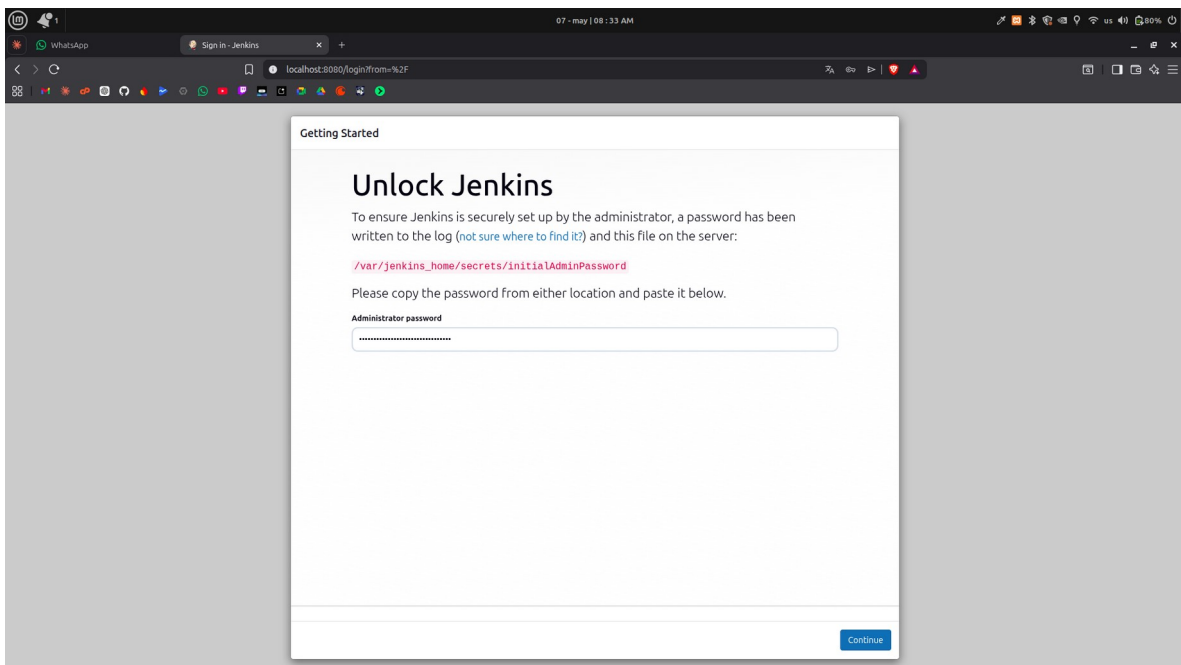
- `docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword`



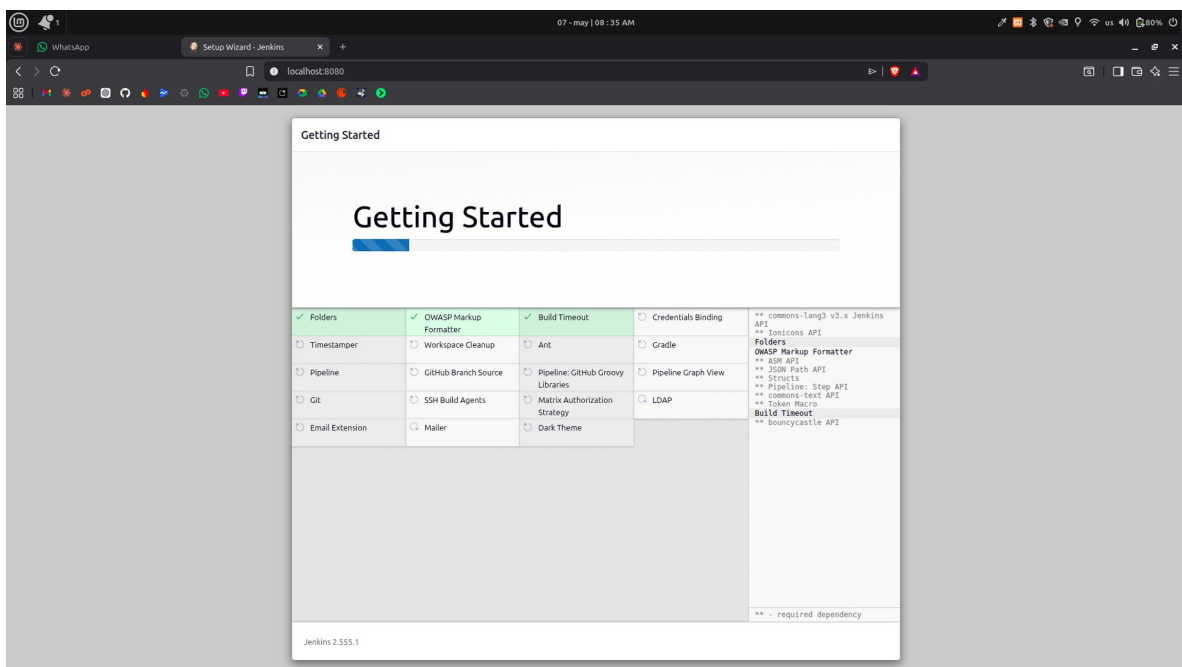
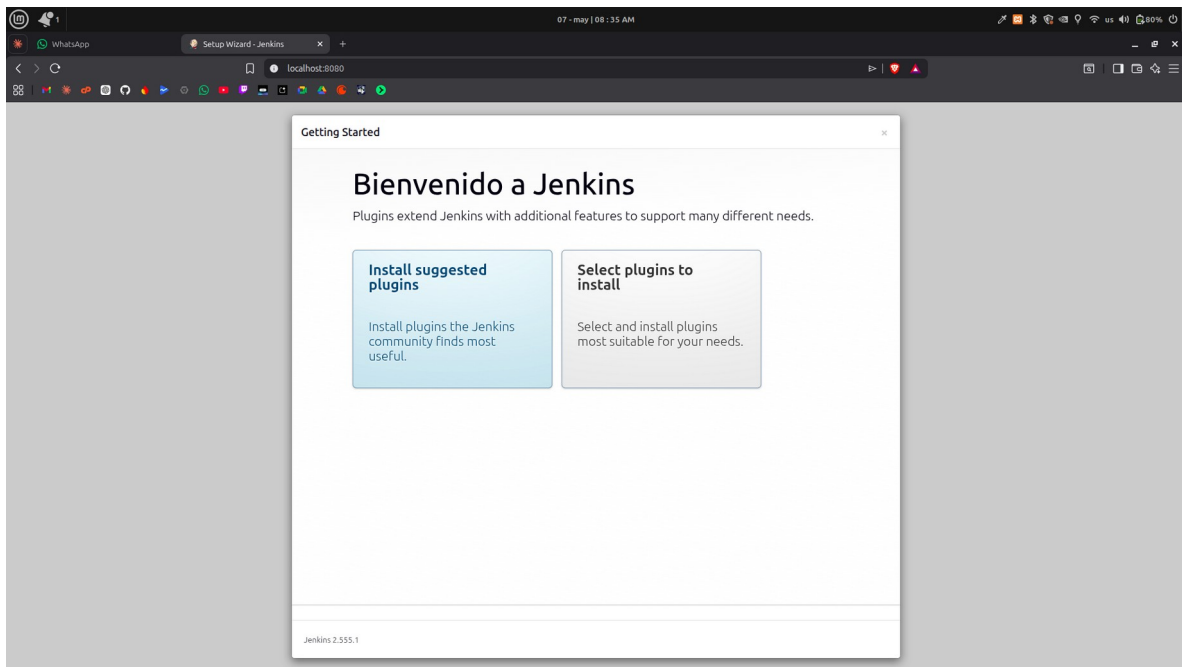
```
miguel@miguel-Dell-G15-5525:~$ docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
44d7f2b2e154fed2c49da1ce65de23c
miguel@miguel-Dell-G15-5525:~$
```

Abrimos Jenkins en el navegador y colocamos esa contraseña:

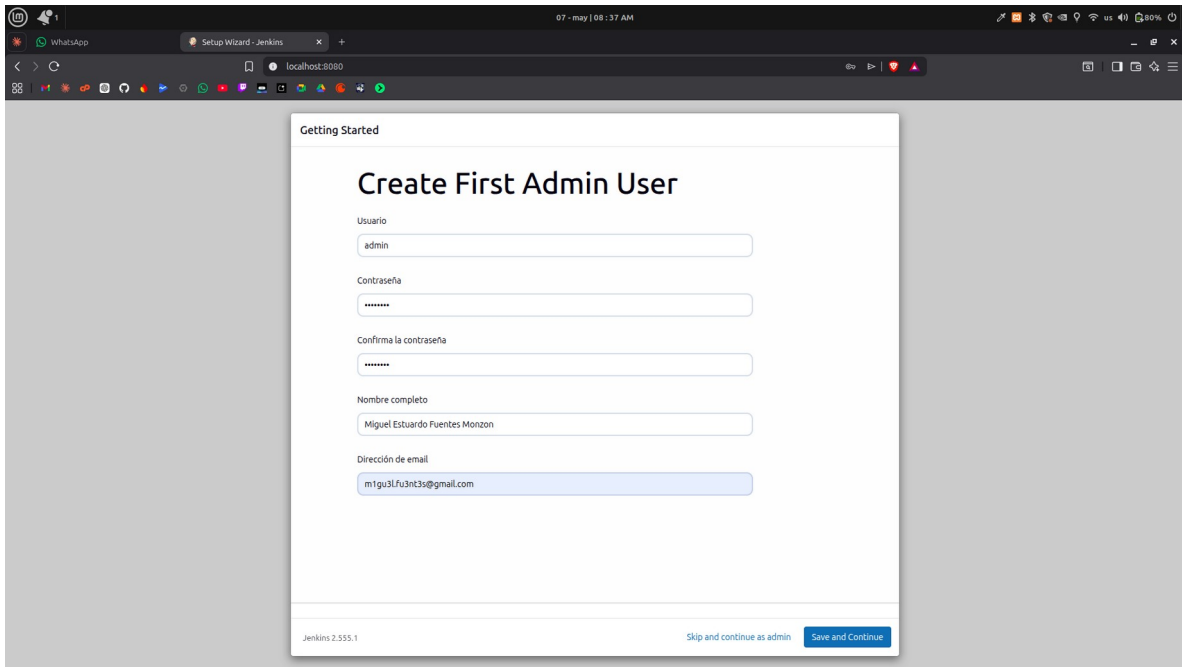
- <http://localhost:8080>



Luego seleccionamos “Install suggested Plugins”:

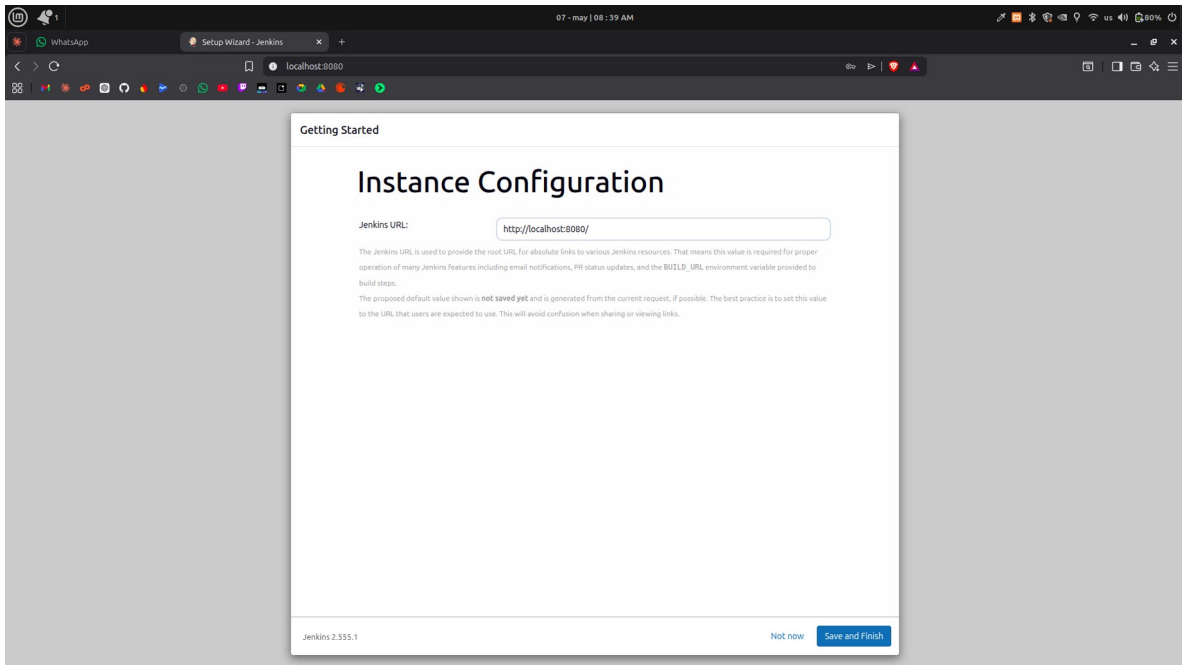


Luego completamos los campos que nos piden:



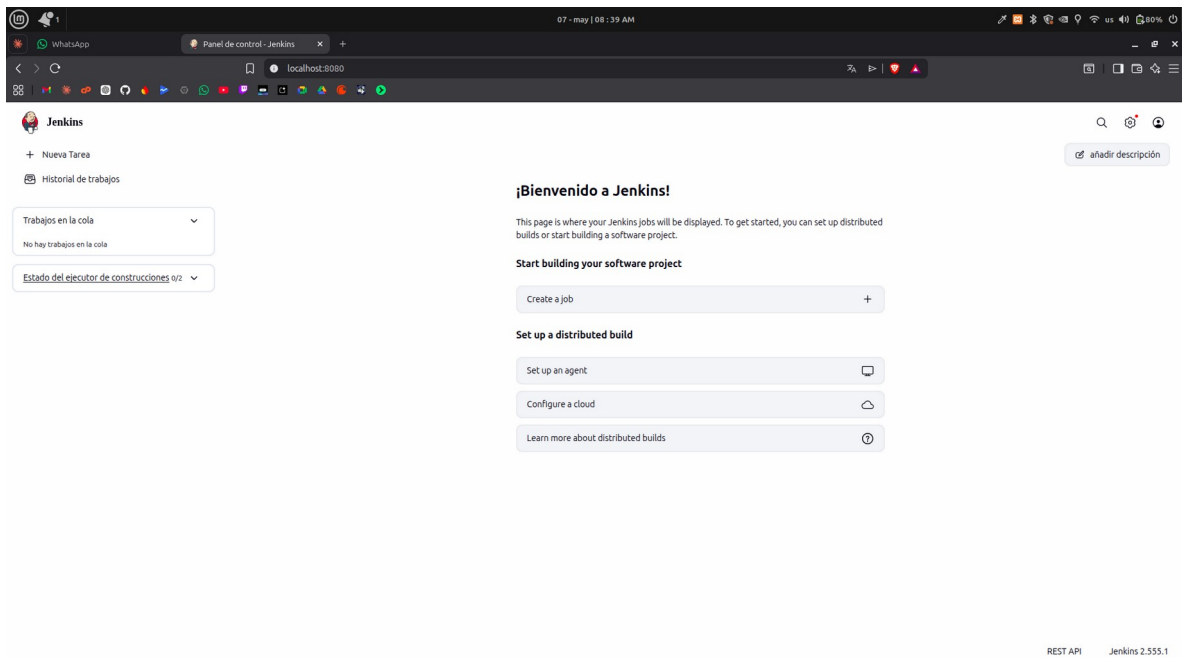
The screenshot shows the 'Getting Started' section of the Jenkins Setup Wizard. The main heading is 'Create First Admin User'. Below this, there are five input fields: 'Usuario' (containing 'admin'), 'Contraseña' (masked with dots), 'Confirma la contraseña' (masked with dots), 'Nombre completo' (containing 'Miguel Estuardo Fuentes Monzon'), and 'Dirección de email' (containing 'm1gu3lFu3n33s@gmail.com'). At the bottom right, there are two buttons: 'Skip and continue as admin' and 'Save and Continue'. The bottom left corner of the wizard indicates 'Jenkins 2.555.1'.

Luego solo le damos “Save and Finish” al otro apartado que nos sale:



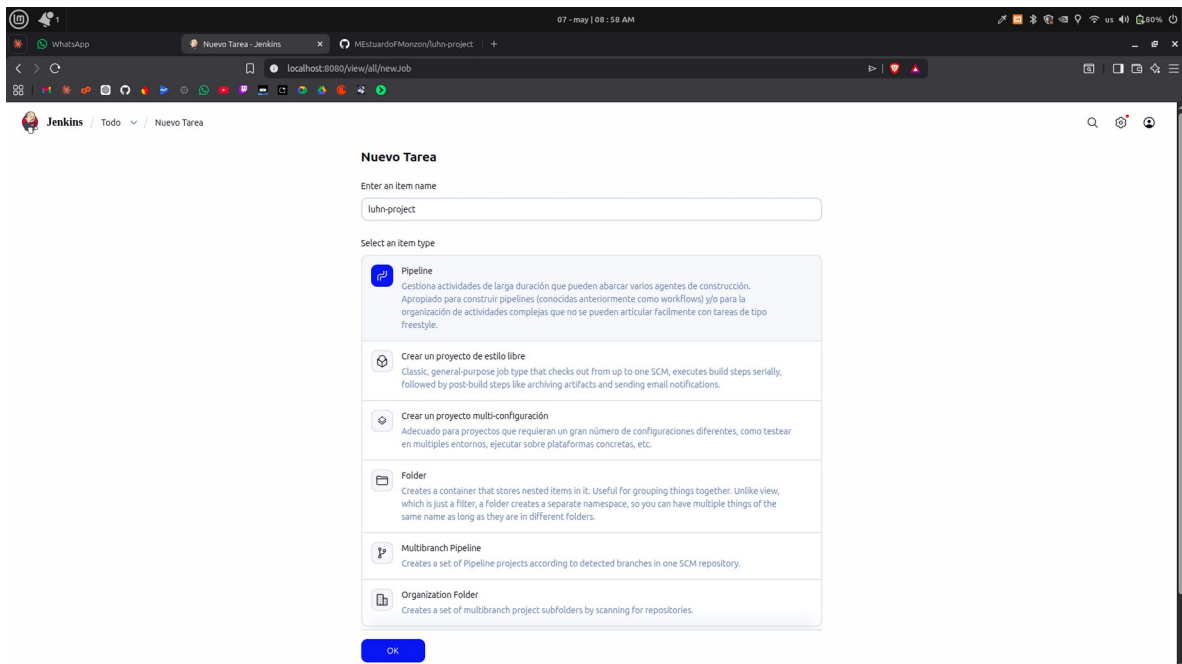
The screenshot shows the 'Getting Started' section of the Jenkins Setup Wizard. The main heading is 'Instance Configuration'. Below this, there is a 'Jenkins URL:' label followed by an input field containing 'http://localhost:8080/'. Below the input field, there is a paragraph of text explaining the purpose of the Jenkins URL. At the bottom right, there are two buttons: 'Not now' and 'Save and Finish'. The bottom left corner of the wizard indicates 'Jenkins 2.555.1'.

Y ya tendríamos lo que es Jenkins:



Paso 3 — Configuración

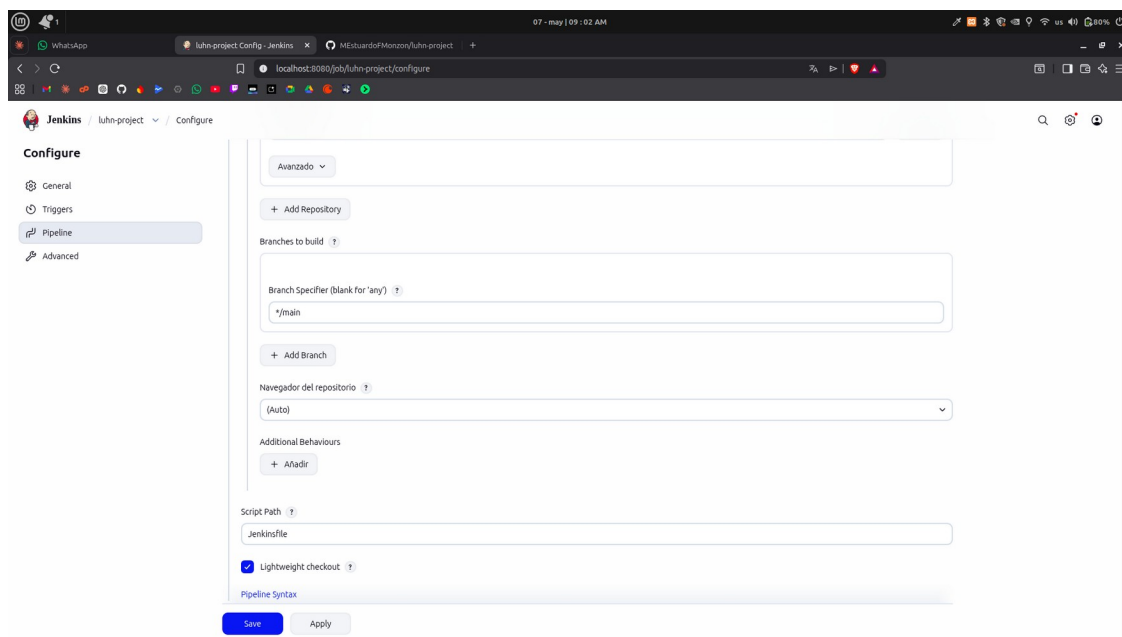
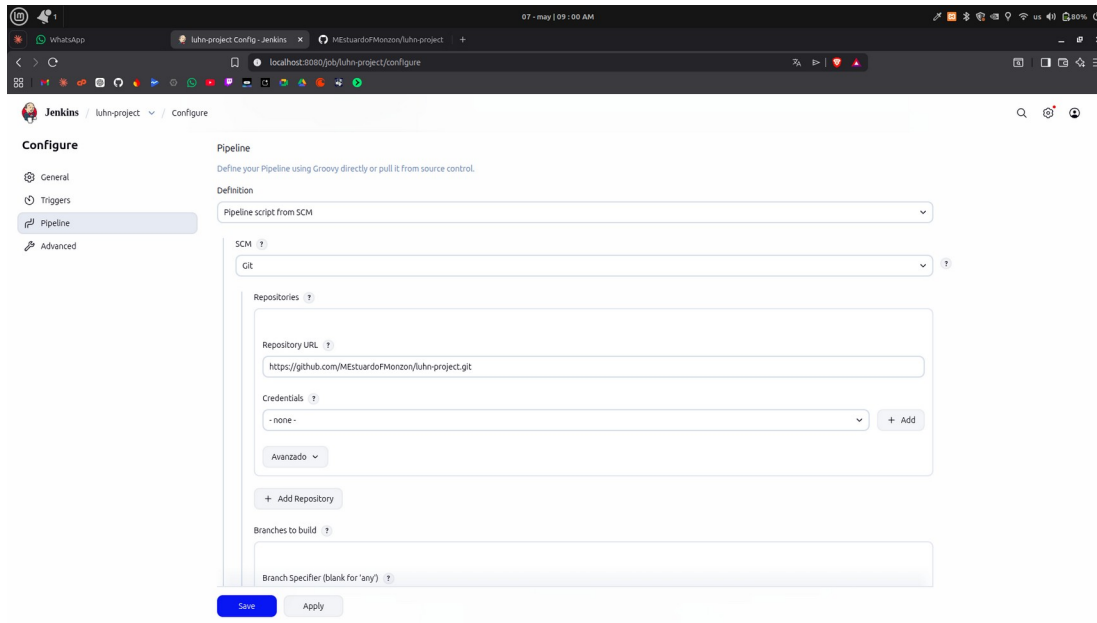
Creamos una nueva tarea en la pagina de Jenkins y seleccionamos unicamente pipeline:



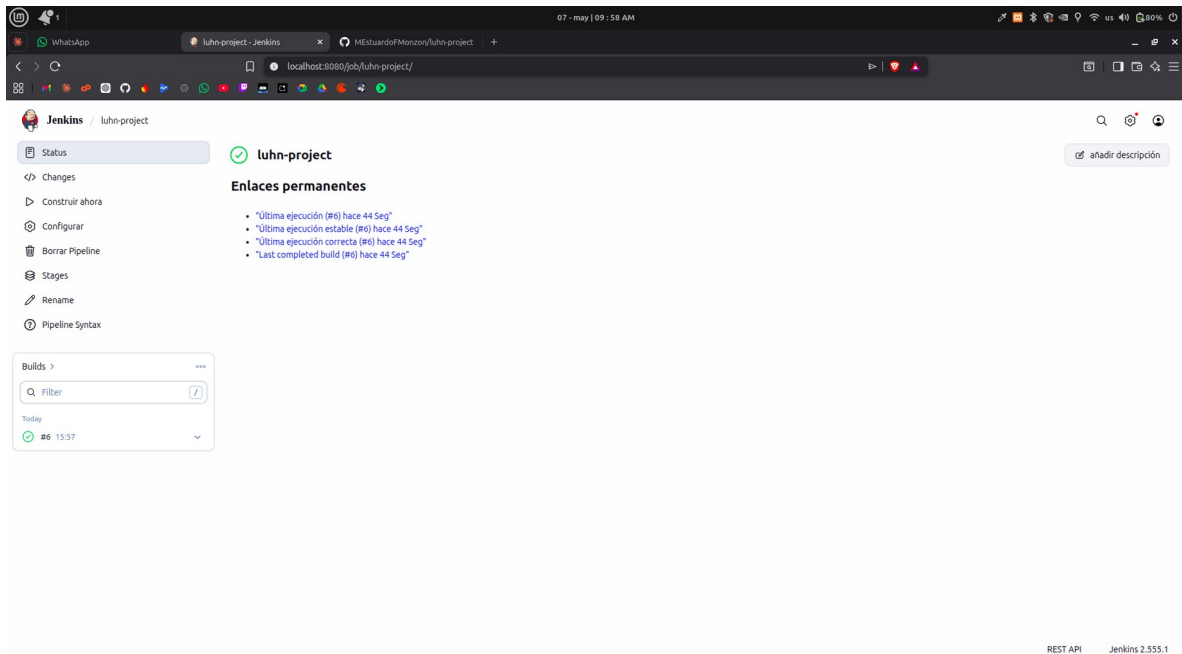
Luego nos vamos al apartado de “Pipeline” y hacemos lo siguiente:

- **En Definition seleccionamos:** Pipeline script from SCM
- **En SCM seleccionamos:** Git
- **En Repository URL pegamos nuestro repositorio:** <https://github.com/MEstuardoFMonzon/luhn-project.git>
- **En Branch Specifier cambia */master por: */main**

Y guardamos



Luego le damos click al apartado de “Construir ahora” para que así podamos ejecutar el Pipeline por primera vez



A mí me dio la primera vez un error porque en el contenedor de Docker donde corre Jenkins no había instalado lo que es npm y tampoco tenía instalado lo que es Node.js, entonces ejecute lo siguiente en la terminal:

- `docker exec -u root -it jenkins bash -c "curl -fsSL https://deb.nodesource.com/setup_20.x | bash - && apt-get install -y nodejs"`
- `docker exec jenkins node --version`
- `docker exec jenkins npm --version`

y ya me devolvió esto la terminal:

```
miguel@miguel-Dell-G15-5525:~$ docker exec jenkins node --version
```

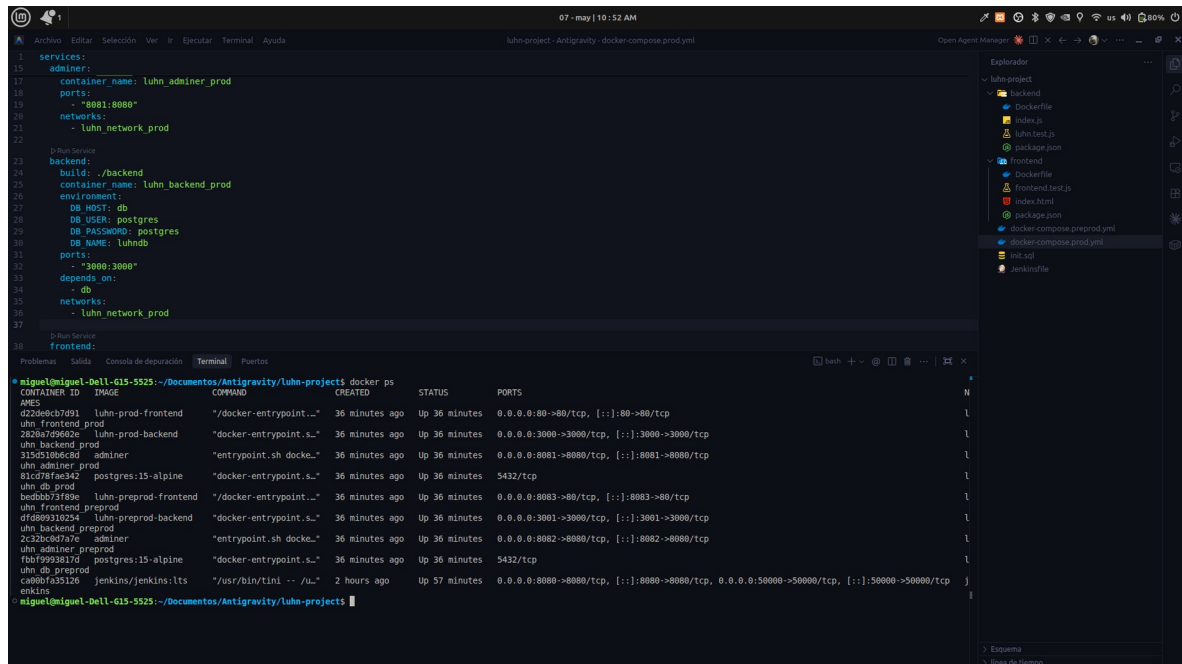
```
v20.20.2
```

```
miguel@miguel-Dell-G15-5525:~$ docker exec jenkins npm --version
```

```
10.8.2
```

```
miguel@miguel-Dell-G15-5525:~$
```

Ahora como ya se ejecuto el pipeline correctamente, volvemos a ejecutar el comando “docker ps” para ver los 8 contenedores, 4 de producción y 4 de preproducción



```
07 - may | 10:52 AM
luhn-project - Antigravity - docker-compose.prod.yml
services:
  adminer:
    container_name: luhn_adminer_prod
    ports:
      - "8081:8080"
    networks:
      - luhn_network_prod
  backend:
    build: ./backend
    container_name: luhn_backend_prod
    environment:
      DB_HOST: db
      DB_USER: postgres
      DB_PASSWORD: postgres
      DB_NAME: luhndb
    ports:
      - "3000:3000"
    depends_on:
      - db
    networks:
      - luhn_network_prod
  frontend:
    build: ./frontend
    container_name: luhn_frontend_prod
    environment:
      DB_HOST: db
      DB_USER: postgres
      DB_PASSWORD: postgres
      DB_NAME: luhndb
    ports:
      - "80:80"
    depends_on:
      - backend
    networks:
      - luhn_network_prod
  frontend-preprod:
    build: ./frontend
    container_name: luhn_frontend_preprod
    environment:
      DB_HOST: db
      DB_USER: postgres
      DB_PASSWORD: postgres
      DB_NAME: luhndb
    ports:
      - "80:80"
    depends_on:
      - backend
    networks:
      - luhn_network_prod
  backend-preprod:
    build: ./backend
    container_name: luhn_backend_preprod
    environment:
      DB_HOST: db
      DB_USER: postgres
      DB_PASSWORD: postgres
      DB_NAME: luhndb
    ports:
      - "3000:3000"
    depends_on:
      - db
    networks:
      - luhn_network_prod
  adminer-preprod:
    container_name: luhn_adminer_preprod
    ports:
      - "8081:8080"
    networks:
      - luhn_network_prod
  db-preprod:
    container_name: luhn_db_preprod
    image: postgres:15-alpine
    ports:
      - "5432:5432"
    networks:
      - luhn_network_prod
  jenkins-preprod:
    container_name: luhn_jenkins_preprod
    image: jenkins/jenkins:lts
    ports:
      - "8080:8080"
    networks:
      - luhn_network_prod
  enkins:
    container_name: enkins
    image: enkins
    ports:
      - "8080:8080"
    networks:
      - luhn_network_prod
networks:
  luhn_network_prod:
```

LINK DEL VIDEO: <https://youtu.be/tJ7LHzpqJmM>

REPOSITORIO: <https://github.com/MEstuardoFMonzon/luhn-project.git>

Analisis Conclusivo

Durante el desarrollo del presente proyecto se logró implementar exitosamente una solución completa de orquestación de contenedores aplicando el Algoritmo de Luhn para la validación de números de identificación, a través de este trabajo se adquirieron conocimientos prácticos sobre herramientas fundamentales en el ámbito de DevOps y administración de sistemas operativos modernos

En primer lugar, se comprendió la importancia de Docker como herramienta de contenedorización, permitiendo empaquetar cada componente del sistema frontend, backend, base de datos y cliente de base de datos y en contenedores independientes y reproducibles, eliminando los problemas de compatibilidad entre entornos y facilitando el despliegue en cualquier sistema operativo

La orquestación mediante Docker Compose demostró ser una solución eficiente para gestionar múltiples servicios de forma coordinada, permitiendo definir en un solo archivo la configuración completa de la infraestructura, y el despliegue en dos ambientes diferenciados producción y preproducción reflejó las buenas prácticas utilizadas en entornos empresariales reales, donde es fundamental probar los cambios antes de llevarlos al ambiente productivo

La integración continua implementada mediante Jenkins permitió automatizar las etapas de pruebas y despliegue del sistema, garantizando que cada cambio en el código fuera validado antes de ser desplegado. Esto demostró el valor del CI/CD como práctica esencial en el desarrollo de software moderno, reduciendo errores humanos y agilizando el ciclo de entrega